

nential fall-off of the integrand makes it behave like a DE rule there too. The mixed rule for $(-\infty, \infty)$ is constructed by splitting the range into $(-\infty, 0)$ and $(0, \infty)$ and making the substitution $x \rightarrow -x$ in the first range. This gives two integrals on $(0, \infty)$.

To implement the DE rule for infinite ranges we don't need the precautions we used in coding the finite range DE rule. It's fine to simply use the routine `Trapzd` directly as a function of t , with the function `funk` that it calls returning $f(x) dx/dt$. So if `funk` is your function returning $f(x)$, then you define the function `func` as a function of t by code of the following form (for the mixed rule)

```
x=exp(t-exp(-t));
dxdt=x*(1.0+exp(-t));
return funk(x)*dxdt;
```

and pass `func` to `Trapzd`. The only care required is in deciding the range of integration. You want the contribution to the integral from the endpoints of the integration to be negligible. For example, $(-4, 4)$ is typically adequate for $x = \exp(\pi \sinh t)$.

4.5.4 Examples

As examples of the power of these methods, consider the following integrals:

$$\int_0^1 \log x \log(1-x) dx = 2 - \frac{\pi^2}{6} \quad (4.5.15)$$

$$\int_0^\infty \frac{1}{x^{1/2}(1+x)} dx = \pi \quad (4.5.16)$$

$$\int_0^\infty x^{-3/2} \sin \frac{x}{2} e^{-x} dx = [\pi(\sqrt{5}-2)]^{1/2} \quad (4.5.17)$$

$$\int_0^\infty x^{-2/7} e^{-x^2} dx = \frac{1}{2} \Gamma\left(\frac{5}{14}\right) \quad (4.5.18)$$

The integral (4.5.15) is easily handled by `DErule`. The routine converges to machine precision (10^{-16}) with about 30 function evaluations, completely unfazed by the singularities at the endpoints. The integral (4.5.16) is an example of an integrand that is singular at the origin and falls off slowly at infinity. The routine `Midinf` fails miserably because of the slow fall-off. Yet the transformation $x = \exp(\pi \sinh t)$ again gives machine precision in about 30 function evaluations, integrating t over the range $(-4, 4)$. By comparison, the transformation $x = e^t$ for t in the range $(-90, 90)$ requires about 500 function evaluations for the same accuracy.

The integral (4.5.17) combines a singularity at the origin with exponential fall-off at infinity. Here the "mixed" transformation $x = \exp(t - e^{-t})$ is best, requiring about 60 function evaluations for t in the range $(-4.5, 4)$. Note that the exponential fall-off is crucial here; these transformations fail completely for slowly decaying oscillatory functions like $x^{-3/2} \sin x$. Fortunately the series acceleration algorithms of §5.3 work well in such cases.

The final integral (4.5.18) is similar to (4.5.17), and using the same transformation requires about the same number of function evaluations to achieve machine precision. The range of t can be smaller, say $(-4, 3)$, because of the more rapid fall-off of the integrand. Note that for all these integrals the number of function evaluations would be double the number we quote if we are using step doubling to

decide when the integrals have converged, since we need one extra set of trapezoidal evaluations to confirm convergence. In many cases, however, you don't need this extra set of function evaluations: Once the method starts converging, the number of significant digits approximately doubles with each iteration. Accordingly, you can set the convergence criterion to stop the procedure when two successive iterations agree to the *square root* of the desired precision. The last iteration will then have approximately the required precision. Even without this trick, the method is quite remarkable for the range of difficult integrals that it can tame efficiently.

An extended example of the use of the DE rule for finite and infinite ranges is given in §6.10. There we give a routine for computing the generalized Fermi-Dirac integrals

$$F_k(\eta, \theta) = \int_0^\infty \frac{x^k (1 + \frac{1}{2}\theta x)^{1/2}}{e^{x-\eta} + 1} dx \quad (4.5.19)$$

Another example is given in the routine `Stiel` in §4.6.

4.5.5 Relation to the Sampling Theorem

The *sinc expansion* of a function is

$$f(x) \simeq \sum_{k=-\infty}^{\infty} f(kh) \operatorname{sinc} \left[\frac{\pi}{h}(x - kh) \right] \quad (4.5.20)$$

where $\operatorname{sinc}(x) \equiv \sin x/x$. The expansion is exact for a limited class of analytic functions. However, it can be a good approximation for other functions too, and the sampling theorem characterizes these functions, as will be discussed in §13.11. There we will use the sinc expansion of e^{-x^2} to get an approximation for the complex error function. Functions well-approximated by the sinc expansion typically fall off rapidly as $x \rightarrow \pm\infty$, so truncating the expansion at $k = \pm N$ still gives a good approximation to $f(x)$.

If we integrate both sides of equation (4.5.20), we find

$$\int_{-\infty}^{\infty} f(x) dx \simeq h \sum_{k=-\infty}^{\infty} f(kh) \quad (4.5.21)$$

which is just the trapezoidal formula! Thus, rapid convergence of the trapezoidal formula for the integral of f corresponds to f being well-approximated by its sinc expansion. The various transformations described earlier can be used to map $x \rightarrow x(t)$ and produce good sinc approximations with uniform samples in t . These approximations can be used not only for the trapezoidal quadrature of f , but also for good approximations to derivatives, integral transforms, Cauchy principal value integrals, and solving differential and integral equations [5].

CITED REFERENCES AND FURTHER READING:

- Schwartz, C. 1969, "Numerical Integration of Analytic Functions," *Journal of Computational Physics*, vol. 4, pp. 19–29.[1]
 Iri, M., Moriguti, S., and Takasawa, Y. 1987, "On a Certain Quadrature Formula," *Journal of Computational and Applied Mathematics*, vol. 17, pp. 3–20. (English version of Japanese article originally published in 1970.)[2]

- Evans, G.A., Forbes, R.C., and Hyslop, J. 1984, "The Tanh Transformation for Singular Integrals," *International Journal of Computer Mathematics*, vol. 15, pp. 339–358.[3]
- Bialecki, B. 1989, *BIT*, "A Modified Sinc Quadrature Rule for Functions with Poles near the Arc of Integration," vol. 29, pp. 464–476.[4]
- Stenger, F. 1981, "Numerical Methods Based on Whittaker Cardinal or Sinc Functions," *SIAM Review*, vol. 23, pp. 165–224.[5]
- Takahasi, H., and Mori, H. 1973, "Quadrature Formulas Obtained by Variable Transformation," *Numerische Mathematik*, vol. 21, pp. 206–219.
- Mori, M. 1985, "Quadrature Formulas Obtained by Variable Transformation and DE Rule," *Journal of Computational and Applied Mathematics*, vol. 12&13, pp. 119–130.
- Sikorski, K., and Stenger, F. 1984, "Optimal Quadratures in H_p Spaces," *ACM Transactions on Mathematical Software*, vol. 10, pp. 140–151; *op. cit.*, pp. 152–160.

4.6 Gaussian Quadratures and Orthogonal Polynomials

In the formulas of §4.1, the integral of a function was approximated by the sum of its functional values at a set of equally spaced points, multiplied by certain aptly chosen weighting coefficients. We saw that as we allowed ourselves more freedom in choosing the coefficients, we could achieve integration formulas of higher and higher order. The idea of *Gaussian quadratures* is to give ourselves the freedom to choose not only the weighting coefficients, but also the location of the abscissas at which the function is to be evaluated. They will no longer be equally spaced. Thus, we will have *twice* the number of degrees of freedom at our disposal; it will turn out that we can achieve Gaussian quadrature formulas whose order is, essentially, twice that of the Newton-Cotes formula with the same number of function evaluations.

Does this sound too good to be true? Well, in a sense it is. The catch is a familiar one, which cannot be overemphasized: High order is not the same as high accuracy. High order translates to high accuracy only when the integrand is very smooth, in the sense of being "well-approximated by a polynomial."

There is, however, one additional feature of Gaussian quadrature formulas that adds to their usefulness: We can arrange the choice of weights and abscissas to make the integral exact for a class of integrands "polynomials times some known function $W(x)$ " rather than for the usual class of integrands "polynomials." The function $W(x)$ can then be chosen to remove integrable singularities from the desired integral. Given $W(x)$, in other words, and given an integer N , we can find a set of weights w_j and abscissas x_j such that the approximation

$$\int_a^b W(x)f(x)dx \approx \sum_{j=0}^{N-1} w_j f(x_j) \quad (4.6.1)$$

is exact if $f(x)$ is a polynomial. For example, to do the integral

$$\int_{-1}^1 \frac{\exp(-\cos^2 x)}{\sqrt{1-x^2}} dx \quad (4.6.2)$$

(not a very natural looking integral, it must be admitted), we might well be interested in a Gaussian quadrature formula based on the choice

$$W(x) = \frac{1}{\sqrt{1-x^2}} \quad (4.6.3)$$

in the interval $(-1, 1)$. (This particular choice is called *Gauss-Chebyshev integration*, for reasons that will become clear shortly.)

Notice that the integration formula (4.6.1) can also be written with the weight function $W(x)$ not overtly visible: Define $g(x) \equiv W(x)f(x)$ and $v_j \equiv w_j / W(x_j)$. Then (4.6.1) becomes

$$\int_a^b g(x)dx \approx \sum_{j=0}^{N-1} v_j g(x_j) \quad (4.6.4)$$

Where did the function $W(x)$ go? It is lurking there, ready to give high-order accuracy to integrands of the form polynomials times $W(x)$, and ready to *deny* high-order accuracy to integrands that are otherwise perfectly smooth and well-behaved. When you find tabulations of the weights and abscissas for a given $W(x)$, you have to determine carefully whether they are to be used with a formula in the form of (4.6.1), or like (4.6.4).

So far our introduction to Gaussian quadrature is pretty standard. However, there is an aspect of the method that is not as widely appreciated as it should be: For smooth integrands (after factoring out the appropriate weight function), Gaussian quadrature converges *exponentially* fast as N increases, because the order of the method, not just the density of points, increases with N . This behavior should be contrasted with the power-law behavior (e.g., $1/N^2$ or $1/N^4$) of the Newton-Cotes based methods in which the order remains fixed (e.g., 2 or 4) even as the density of points increases. For a more rigorous discussion, see §20.7.4.

Here is an example of a quadrature routine that contains the tabulated abscissas and weights for the case $W(x) = 1$ and $N = 10$. Since the weights and abscissas are, in this case, symmetric around the midpoint of the range of integration, there are actually only five distinct values of each:

```
qgaus.h  template <class T>
          Doub qgaus(T &func, const Doub a, const Doub b)
Returns the integral of the function or functor func between a and b, by ten-point Gauss-
Legendre integration: the function is evaluated exactly ten times at interior points in the range
of integration.
{
    Here are the abscissas and weights:
    static const Doub x[]={0.1488743389816312,0.4333953941292472,
        0.6794095682990244,0.8650633666889845,0.9739065285171717};
    static const Doub w[]={0.2955242247147529,0.2692667193099963,
        0.2190863625159821,0.1494513491505806,0.0666713443086881};
    Doub xm=0.5*(b+a);
    Doub xr=0.5*(b-a);
    Doub s=0;
    for (Int j=0;j<5;j++) {
        Doub dx=xr*x[j];
        s += w[j]*(func(xm+dx)+func(xm-dx));
    }
    return s * xr;
}
Will be twice the average value of the function, since the
ten weights (five numbers above each used twice)
sum to 2.
Scale the answer to the range of integration.
```

The above routine illustrates that one can use Gaussian quadratures without necessarily understanding the theory behind them: One just locates tabulated weights and abscissas in a book (e.g., [1] or [2]). However, the theory is very pretty, and it will come in handy if you ever need to construct your own tabulation of weights and abscissas for an unusual choice of $W(x)$. We will therefore give, without any proofs, some useful results that will enable you to do this. Several of the results assume that $W(x)$ does not change sign inside (a, b) , which is usually the case in practice.

The theory behind Gaussian quadratures goes back to Gauss in 1814, who used continued fractions to develop the subject. In 1826, Jacobi rederived Gauss's results by means of orthogonal polynomials. The systematic treatment of arbitrary weight functions $W(x)$ using orthogonal polynomials is largely due to Christoffel in 1877. To introduce these orthogonal polynomials, let us fix the interval of interest to be (a, b) . We can define the "scalar product of two functions f and g over a weight function W " as

$$\langle f | g \rangle \equiv \int_a^b W(x) f(x) g(x) dx \quad (4.6.5)$$

The scalar product is a number, not a function of x . Two functions are said to be *orthogonal* if their scalar product is zero. A function is said to be *normalized* if its scalar product with itself is unity. A set of functions that are all mutually orthogonal and also all individually normalized is called an *orthonormal set*.

We can find a set of polynomials (i) that includes exactly one polynomial of order j , called $p_j(x)$, for each $j = 0, 1, 2, \dots$, and (ii) all of which are mutually orthogonal over the specified weight function $W(x)$. A constructive procedure for finding such a set is the recurrence relation

$$\begin{aligned} p_{-1}(x) &\equiv 0 \\ p_0(x) &\equiv 1 \\ p_{j+1}(x) &= (x - a_j) p_j(x) - b_j p_{j-1}(x) \quad j = 0, 1, 2, \dots \end{aligned} \quad (4.6.6)$$

where

$$\begin{aligned} a_j &= \frac{\langle x p_j | p_j \rangle}{\langle p_j | p_j \rangle} \quad j = 0, 1, \dots \\ b_j &= \frac{\langle p_j | p_j \rangle}{\langle p_{j-1} | p_{j-1} \rangle} \quad j = 1, 2, \dots \end{aligned} \quad (4.6.7)$$

The coefficient b_0 is arbitrary; we can take it to be zero.

The polynomials defined by (4.6.6) are *monic*, that is, the coefficient of their leading term $[x^j \text{ for } p_j(x)]$ is unity. If we divide each $p_j(x)$ by the constant $[\langle p_j | p_j \rangle]^{1/2}$, we can render the set of polynomials orthonormal. One also encounters orthogonal polynomials with various other normalizations. You can convert from a given normalization to monic polynomials if you know that the coefficient of x^j in p_j is λ_j , say; then the monic polynomials are obtained by dividing each p_j by λ_j . Note that the coefficients in the recurrence relation (4.6.6) depend on the adopted normalization.

The polynomial $p_j(x)$ can be shown to have exactly j distinct roots in the interval (a, b) . Moreover, it can be shown that the roots of $p_j(x)$ "interleave" the $j - 1$ roots of $p_{j-1}(x)$, i.e., there is exactly one root of the former in between each two adjacent roots of the latter. This fact comes in handy if you need to find all the